

Projets pour étudiants.

Projet 1: Système Intelligent de Gestion des Files d'Attente dans un Hôpital

Problème réel

Dans de nombreux hôpitaux, les patients arrivent et attendent parfois plusieurs heures sans savoir quand ils seront reçus, ils ne savent pas combien de personnes sont déjà devant eux, ils passent parfois toute la journée à attendre.

Cette situation provoque :

- frustration des patients ;
- encombrement des salles d'attente ;
- perte de temps;
- conflits liés au non-respect de l'ordre d'arrivée ;
- mauvaise organisation des urgences.

L'objectif du projet est de développer un logiciel permettant d'organiser automatiquement les consultations.

Nouvelle idée

Le patient s'enregistre depuis chez lui :

- téléphone
- ordinateur
- cybercafé
- application mobile

Le système lui indique :

"Ne venez pas maintenant. Revenez à 13h40."

Fonctionnement

Étape 1 : Inscription à distance

Le patient saisit :

- nom
- âge
- sexe
- service demandé
- symptômes

Exemple :

Nom : Koffi

Service : Cardiologie

Symptômes :

- douleurs thoraciques
- essoufflement

Étape 2 : Vérification automatique

Au lieu de demander :

Choisissez votre priorité :

- ☐ Faible
- ☐ Moyenne

☐ Élevée

ce qui est absurde car tout le monde choisirait "élevée",
le système détermine lui-même la priorité.

Détermination automatique de la priorité

Le patient répond à un questionnaire.

Exemple :

Cas cardiologie

Avez-vous :

☐ douleur thoracique sévère ?

☐ perte de connaissance ?

☐ difficulté à respirer ?

☐ Hypertension connue ?

Le système attribue des points.

Exemple :

Symptôme	Points
Fièvre	1
Toux	1
Douleur modérée	2
Femme enceinte	3
Difficulté respiratoire	8
Perte de connaissance	10

Score :

0 - 3 Faible

4 - 7 Moyenne

8 - 12 Élevée

>12 Urgence

Ainsi :

- le patient ne choisit jamais sa priorité ;
- c'est l'algorithme qui la calcule.

Étape 3 : Gestion de la capacité du médecin

Supposons :

Médecin :

Consultation moyenne :

15 minutes

Patients déjà enregistrés :

08h00
08h15
08h30
08h45
09h00

Un nouveau patient arrive.
Le système voit :
5 personnes déjà programmées

et calcule :
Heure probable :

09h15

Étape 4 : Heure recommandée

Le patient reçoit :
Votre consultation est prévue à :

09h15

Veuillez arriver 10 minutes avant.

Ainsi :

- il reste chez lui ;
- il évite des heures d'attente ;
- l'hôpital est moins encombré.

Gestion des retards

Problème fréquent en Afrique :
Le patient reçoit :
09h15

mais arrive à : 11h00
Le système doit avoir une règle :
Retard > 10 min :
Rendez-vous annulé

ou
Patient replacé en fin de file.

Gestion des urgences

Certaines personnes ne peuvent pas attendre.
Exemples :

- accident
- AVC

- crise cardiaque

Ces patients vont directement aux urgences.

Ils ne passent jamais par l'application.

Ainsi :

- les urgences réelles restent prioritaires ;
- le système gère uniquement les consultations planifiées.

Celui-ci est un **excellent projet de Licence 3**, car il résout un problème que presque tout le monde rencontre en Afrique : les coupures d'électricité imprévisibles.

Mais pour que le projet soit vraiment utile, il ne faut pas simplement enregistrer les coupures. Il faut que le système soit capable de **prédire et aider les utilisateurs à s'organiser**.

Projet 2 : SmartPower Africa

Système Intelligent de Gestion et de Prévision des Coupures d'Électricité

Contexte

Dans de nombreuses villes africaines, les coupures d'électricité :

- perturbent le travail ;
- empêchent les étudiants d'étudier ;
- interrompent les activités commerciales ;
- endommagent parfois les équipements électriques.

Le problème principal est que les habitants ne savent généralement pas :

- quand la coupure va commencer ;
- combien de temps elle va durer ;
- quel est le meilleur moment pour lancer certaines activités.

Objectif général

Développer une application capable de :

- enregistrer les coupures observées ;
- construire un historique ;
- détecter les habitudes de coupure ;
- estimer les futures coupures ;
- envoyer des alertes ;
- recommander les meilleurs créneaux horaires.

Fonctionnement général

Écran principal

SMARTPOWER AFRICA

1. Déclarer une coupure
2. Déclarer un retour du courant
3. Consulter l'historique
4. Prévisions
5. Statistiques
6. Alertes
7. Quitter

Partie 1 : Déclaration d'une coupure

Lorsque le courant disparaît :

L'utilisateur clique :

Déclarer une coupure

Le système enregistre automatiquement :

Date

Heure

Quartier

Ville

Exemple :

15/06/2026

14:25

Quartier :

Agoè

Partie 2 : Déclaration du retour du courant

Lorsque le courant revient :

Retour du courant

Le système calcule :

Durée = Heure retour - Heure coupure

Exemple :

Début : 14:25

Retour : 16:40

Durée : 2 h 15 min

Base de données SQLite

Table :

CREATE TABLE coupures (

id INTEGER PRIMARY KEY,

date_coupure TEXT,

heure_coupure TEXT,

heure_retour TEXT,

duree REAL,

quartier TEXT

);

Partie 3 : Historique

L'utilisateur peut consulter :

Date	Début	Fin	Durée
-------------	--------------	------------	--------------

15/06	14:25	16:40	2h15
-------	-------	-------	------

14/06	10:05	11:00	55 min
-------	-------	-------	--------

13/06	18:10	20:30	2h20
-------	-------	-------	------

Partie 4 : Analyse automatique

Le logiciel calcule :

Nombre de coupures

Ce mois :

23 coupures

Durée moyenne

1 h 42 min

Jour le plus perturbé

Mardi

Heure critique

Entre 18 h et 21 h

Partie 5 : Prévision intelligente

Supposons l'historique suivant :

18h10

18h20

18h15

18h05

18h18

Le système détecte que :

La plupart des coupures
commencent vers 18h.

Il affiche :

Risque élevé de coupure
entre 18h00 et 18h30.

Partie 6 : Recommandation d'activités

Le système aide l'utilisateur.

Exemple :

L'utilisateur veut :

Étudier 3 heures

Le logiciel consulte l'historique.

Il répond :

Créneau conseillé :

09h00 - 12h00

Autre exemple :

Utilisation machine à laver

Le système répond :

Période recommandée :

07h00 - 11h00

Partie 7 : Carte des quartiers

Version avancée.

Les utilisateurs d'un même quartier enregistrent leurs coupures.

Le système calcule :

Quartier A :

15 coupures

Quartier B :

8 coupures

Quartier C :

3 coupures

Classement automatique

Quartiers les plus touchés

1. Agoè

2. Bè

3. Adidogomé

Partie 8 : Alertes

Si le système détecte :

Probabilité élevée de coupure
entre 18h et 19h

Il envoie :
Attention

Une coupure est probable
dans moins d'une heure.

Extension WhatsApp / SMS

Les meilleurs groupes peuvent connecter :

- SMS
- WhatsApp

Exemple :
SMARTPOWER

Risque élevé de coupure
à partir de 18h15.

Pensez à charger vos appareils.

Interface graphique

Avec Tkinter :

Onglet 1

Déclarer une coupure.

Onglet 2

Déclarer le retour.

Onglet 3

Historique.

Onglet 4

Prévisions.

Onglet 5

Statistiques.

PROJET 3: Analyse des propriétés optiques des matériaux à partir de la fonction diélectrique

Contexte

En science des matériaux, les propriétés optiques permettent de comprendre comment un matériau interagit avec la lumière. Elles sont essentielles dans :

- les semi-conducteurs
- les cellules photovoltaïques
- les matériaux nano-structurés
- l'optoélectronique

Ces propriétés sont généralement obtenues à partir de la fonction diélectrique complexe :

$$\epsilon(\omega) = \epsilon_1(\omega) + i \epsilon_2(\omega)$$

Objectif du projet

Développer un programme Python permettant de calculer et analyser les propriétés optiques d'un matériau à partir de sa fonction diélectrique.

Données d'entrée

Les étudiants doivent chercher eux-mêmes des fichiers de données pour environ 5 matériaux, contenant :

- l'énergie ou fréquence (ω ou E en eV)
- ϵ_1 (partie réelle)
- ϵ_2 (partie imaginaire)

Format attendu : fichier CSV ou texte structuré.

Travail demandé

1. Calcul des propriétés optiques

À partir de ϵ_1 et ϵ_2 , calculer les grandeurs suivantes :

♦ Module de la fonction diélectrique

$$|\epsilon| = \sqrt{\epsilon_1^2 + \epsilon_2^2}$$

♦ Indice de réfraction

$$n(\omega) = \sqrt{(|\epsilon| + \epsilon_1) / 2}$$

♦ Coefficient d'extinction

$$k(\omega) = \sqrt{(|\epsilon| - \epsilon_1) / 2}$$

♦ Réflectivité

$$R(\omega) = ((n - 1)^2 + k^2) / ((n + 1)^2 + k^2)$$

♦ Coefficient d'absorption

$$\alpha(\omega) = (2 \omega k) / c$$

où :

- $c = 3 \times 10^8$ m/s
- ω = fréquence angulaire

2. Représentation graphique

Tracer en fonction de l'énergie :

- $\epsilon_1(E)$ et $\epsilon_2(E)$
- $n(E)$
- $k(E)$

- $R(E)$
- $\alpha(E)$

Toutes les courbes doivent être claires, annotées et lisibles.

3. Analyse physique

Les étudiants doivent interpréter les résultats en répondant aux questions suivantes :

- Le matériau est-il plutôt conducteur, semi-conducteur ou isolant ?
- Dans quelle gamme d'énergie le matériau est-il transparent ?
- Où observe-t-on des pics d'absorption ?
- Quel est le comportement optique global du matériau ?

Une version avancée voudrait que vous construisiez l'application de sorte qu'il puisse lui-même faire les analyses physiques.

4. Comparaison de matériaux

Le programme doit pouvoir analyser plusieurs matériaux et permettre :

- comparaison des courbes optiques
- identification du matériau le plus absorbant
- identification du matériau le plus transparent

Notions physiques utilisées

Les étudiants doivent utiliser les relations suivantes :

Fonction diélectrique complexe :

$$\varepsilon(\omega) = \varepsilon_1(\omega) + i \varepsilon_2(\omega)$$

Module :

$$|\varepsilon| = \sqrt{\varepsilon_1^2 + \varepsilon_2^2}$$

Indice de réfraction :

$$n(\omega) = \sqrt{(|\varepsilon| + \varepsilon_1) / 2}$$

Coefficient d'extinction :

$$k(\omega) = \sqrt{(|\varepsilon| - \varepsilon_1) / 2}$$

Réflexivité :

$$R(\omega) = ((n - 1)^2 + k^2) / ((n + 1)^2 + k^2)$$

Absorption :

$$\alpha(\omega) = (2 \omega k) / c$$

avec :

$$c = 3 \times 10^8 \text{ m/s}$$

PROJET 4: Simulation numérique d'un système de transport d'énergie électrique dans le vide par faisceau d'électrons guidé par des champs électromagnétiques

Résumé

L'objectif de ce projet est de concevoir un système capable de transporter de l'énergie électrique dans une chambre sous vide sous la forme d'un faisceau d'électrons. Les électrons sont émis par une cathode, accélérés par un champ électrique, guidés et focalisés par des bobines magnétiques, puis collectés par une anode afin de reconstituer un courant électrique exploitable.

Le projet consiste à développer un simulateur Python permettant de modéliser les champs électriques et magnétiques, le mouvement des électrons, les pertes, l'efficacité énergétique et l'optimisation de la géométrie du système.

Objectif général

Développer un simulateur Python capable de modéliser le transport d'un faisceau d'électrons dans le vide.

Objectifs spécifiques

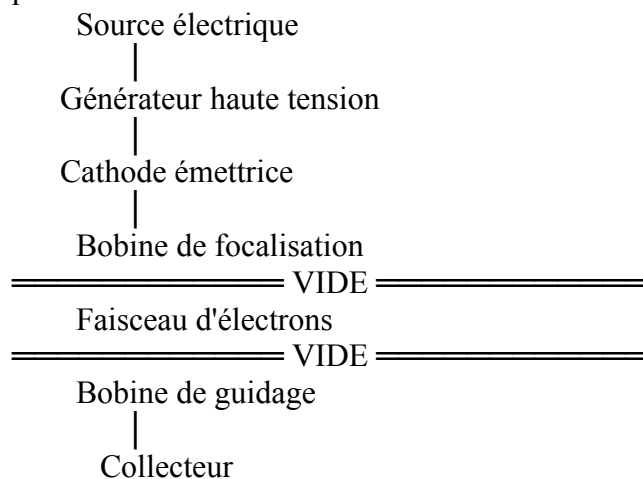
- Modéliser l'émission électronique.
- Résoudre l'équation de Poisson.
- Calculer le champ magnétique produit par des bobines.
- Simuler les trajectoires électroniques.
- Étudier le guidage magnétique.
- Calculer les pertes.
- Optimiser le rendement.
- Comparer différentes géométries.

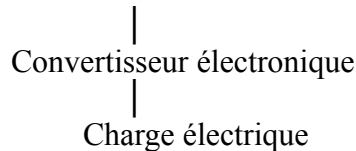
Applications

Le système étudié est proche des technologies utilisées dans :

- tubes électroniques,
- microscopes électroniques,
- accélérateurs de particules,
- propulsion ionique,
- tubes à ondes progressives,
- systèmes de conversion d'énergie sous vide.

Le projet est également pertinent pour la recherche fondamentale sur le transport de faisceaux de particules.





Théorie

1. Émission électronique

Selon le type de cathode, plusieurs mécanismes sont possibles :

Émission thermoïonique

Loi de Richardson-Dushman $J = AT^2 e^{-\phi/kT}$

avec : A : constante de Richardson, T : température, ϕ : travail d'extraction.

Émission de champ

Loi de Fowler-Nordheim $J = aE^2 e^{-b/E}$ où E est le champ électrique à la surface de la cathode.

2. Potentiel électrique

Équation de Poisson $\nabla^2 V = -\rho/\epsilon_0$ puis $E = -\nabla V$

3. Champ magnétique

Les bobines sont modélisées par la loi de Biot-Savart ou, pour un solénoïde long : $B = \mu_0 n I$ où n est le nombre de spires par unité de longueur.

4. Mouvement des électrons

Force de Lorentz $F = q(E + v \wedge B)$

Les équations du mouvement sont : $m dv/dt = q(E + v \wedge B)$ et $dr/dt = v$

5. Charge d'espace

Lorsque le faisceau devient dense, les électrons se repoussent mutuellement. On peut utiliser la loi de Child-Langmuir pour estimer le courant maximal dans une diode plane :

6. Rendement énergétique

Le rendement global peut être défini par :

avec

$$J = \frac{4}{9} \epsilon_0 \sqrt{\frac{2e}{m}} \frac{V^{3/2}}{d^2}$$

Algorithme

1. Définir la géométrie.
2. Calculer le potentiel électrique.
3. Calculer le champ électrique.
4. Calculer le champ magnétique.
5. Émettre des électrons.
6. Intégrer leurs trajectoires (par exemple avec un schéma de Boris, très utilisé pour les particules chargées).
7. Détecter les électrons collectés.
8. Calculer le courant.
9. Évaluer le rendement.
10. Optimiser les paramètres.

Résultats attendus

Le simulateur pourra produire :

- le potentiel électrique ;
- les lignes de champ électrique ;
- les lignes de champ magnétique ;
- les trajectoires 3D des électrons ;
- la vitesse et l'énergie des électrons ;
- le courant collecté ;
- le rendement énergétique ;

$$\eta = \frac{P_{\text{collectée}}}{P_{\text{injectée}}}$$

$$P = VI$$

- l'effet de la tension, de la géométrie des bobines et du vide sur les performances.
- d'un rapport scientifique présentant la méthodologie, les résultats et les perspectives

PROJET 5: Modélisation prédictive de la dégradation des batteries lithium-ion par Python

1. Contexte

Les batteries lithium-ion sont aujourd'hui les principales sources d'énergie utilisées dans les véhicules électriques, les smartphones, les ordinateurs portables et les systèmes de stockage des énergies renouvelables.

Cependant, leurs performances diminuent progressivement avec les cycles de charge et de décharge. Cette dégradation entraîne une réduction de la capacité, une augmentation de la résistance interne et une diminution de leur durée de vie.

Grâce aux outils numériques et au Machine Learning, il est aujourd'hui possible de prédire cette dégradation afin d'optimiser l'utilisation des batteries et de réduire les coûts de maintenance.

Ce projet permettra aux étudiants de développer un modèle prédictif sous Python.

2. Problématique

Comment utiliser les données expérimentales d'une batterie lithium-ion pour prédire son vieillissement et estimer sa durée de vie restante ?

3. Objectif général

Développer un programme Python capable de prédire la dégradation d'une batterie lithium-ion en fonction des cycles de charge/décharge.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre le fonctionnement d'une batterie lithium-ion ;
- étudier les mécanismes de vieillissement ;
- collecter ou utiliser une base de données expérimentale ;
- nettoyer les données ;
- visualiser l'évolution de la capacité ;
- développer plusieurs modèles prédictifs ;
- comparer leurs performances ;
- estimer la durée de vie restante (Remaining Useful Life : RUL)

6. Notions théoriques

Les étudiants étudieront notamment :

Les batteries lithium-ion

- cathode
- anode
- électrolyte
- séparateur

Les phénomènes de dégradation

- perte de lithium actif
- croissance de la couche SEI
- fissuration des électrodes
- augmentation de la résistance interne
- perte de capacité

Les indicateurs de santé

State of Charge (SOC)

$SOC = Q_{\text{restant}} / Q_{\text{nominal}} \times 100$

State of Health (SOH)

$SOH = \text{Capacité actuelle} / \text{Capacité initiale} \times 100$

7. Méthodologie

Étape 1

Recherche bibliographique.

Étape 2

Téléchargement des données expérimentales.

Exemples : NASA Battery Dataset, CALCE Battery Dataset, et Oxford Battery Degradation Dataset

Les données comprennent généralement : numéro du cycle, capacité, tension, courant, température, et résistance interne

Étape 4

Visualisation

Tracer : capacité vs cycles, température vs cycles, résistance interne et la tension

Étape 5

Construction des modèles

Les étudiants comparent plusieurs méthodes.

Modèle 1

Régression linéaire (Prédire la capacité restante.)

Modèle 2

Random Forest (Pour améliorer la précision.)

Modèle 3

Gradient Boosting (Très performant sur les données non linéaires.)

Modèle 4

Réseau de neurones (Pour apprendre automatiquement les relations complexes.)

8. Résultats attendus

À la fin du projet, les étudiants disposeront :

- d'un programme Python fonctionnel ;
- d'un modèle de prédiction du vieillissement ;
- d'un rapport scientifique présentant la méthodologie, les résultats et les perspectives

PROJECT 6: Simulation multiphysique d'un système photovoltaïque-électrolyseur pour production d'hydrogène

1. Contexte

La transition énergétique mondiale repose sur le développement de technologies capables de produire une énergie propre et durable. L'hydrogène vert est considéré comme un vecteur énergétique prometteur, car il peut être produit à partir de l'eau en utilisant de l'électricité issue de sources renouvelables telles que le solaire photovoltaïque.

Un système photovoltaïque-électrolyseur convertit l'énergie solaire en électricité grâce aux panneaux photovoltaïques, puis utilise cette électricité pour alimenter un électrolyseur qui décompose l'eau en hydrogène et en oxygène. Les performances globales de ce système

dépendent des interactions entre plusieurs phénomènes physiques : optiques, électriques, thermiques et électrochimiques.

Ce projet vise à développer un simulateur numérique sous Python permettant d'étudier le comportement multiphysique d'un système photovoltaïque–électrolyseur et d'optimiser sa production d'hydrogène.

2. Problématique

Comment modéliser numériquement le fonctionnement couplé d'un panneau photovoltaïque et d'un électrolyseur afin d'optimiser la production d'hydrogène vert sous différentes conditions climatiques ?

3. Objectif général

Développer un modèle multiphysique sous Python permettant de simuler les performances d'un système photovoltaïque–électrolyseur destiné à la production d'hydrogène.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre le fonctionnement d'un panneau photovoltaïque ;
- étudier les principes de l'électrolyse de l'eau ;
- modéliser la production électrique d'un module photovoltaïque ;
- modéliser les performances d'un électrolyseur ;
- coupler les deux modèles ;
- analyser l'influence de l'irradiance solaire et de la température ;
- estimer la quantité d'hydrogène produite ;
- optimiser les paramètres du système.

6. Théorie

6.1 Énergie photovoltaïque

Le panneau photovoltaïque transforme le rayonnement solaire en énergie électrique.

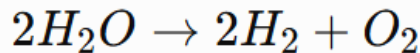
Les principaux paramètres sont : irradiance solaire (W/m^2) ; température des cellules ; tension courant ; puissance ; rendement.

Le modèle à une diode peut être utilisé :

$$I = I_{ph} - I_0 \left(e^{\frac{q(V+IR_s)}{nkT}} - 1 \right) - \frac{V + IR_s}{R_{sh}}$$

6.2 Électrolyse de l'eau

Réaction globale :



Deux technologies peuvent être étudiées : électrolyseur alcalin ; électrolyseur PEM.

6.3 Loi de Faraday

La quantité d'hydrogène produite est donnée par :

$$m_{H_2} = \frac{\eta_F M I t}{2F}$$

où : M = masse molaire de l'hydrogène ; F = constante de Faraday ; I = courant ; t = temps ; η_F = rendement faradique.

6.4 Rendement global

$$\eta_{global} = \eta_{PV} \times \eta_{electrolyseur}$$

6.5 Influence de la température

Les étudiants étudieront : diminution du rendement photovoltaïque ; augmentation de la cinétique électrochimique ; compromis thermique.

7. Méthodologie

Étape 1

Recherche bibliographique.

Étape 2

Modélisation du panneau photovoltaïque.

Simulation de : courbe I-V ; courbe P-V ; puissance maximale.

Étape 3

Modélisation de l'électrolyseur.

Calcul : tension de cellule ; rendement ; consommation électrique ; production d'hydrogène.

Étape 4

Couplage des deux modèles.

Le courant produit par le panneau devient l'entrée de l'électrolyseur.

Étape 5

Simulation sous différentes conditions.

Les étudiants feront varier : irradiance ; température ; surface photovoltaïque ; nombre de cellules ; pression (optionnelle).

PROJET 7: Cartographie des zones inondables

1. Contexte

Les inondations figurent parmi les catastrophes naturelles les plus fréquentes et les plus destructrices dans de nombreuses régions du monde, notamment en Afrique de l'Ouest (cas du Togo). Elles entraînent des pertes humaines, des dégâts matériels importants et des impacts économiques considérables.

Grâce aux **Systèmes d'Information Géographique (SIG)**, à la **télédétection** et à la **modélisation numérique**, il est aujourd'hui possible de cartographier les zones les plus exposées aux inondations. Ces cartes constituent des outils d'aide à la décision pour les collectivités, les services de protection civile et les urbanistes.

Ce projet propose de développer une application Python capable d'identifier et de cartographier les zones présentant un risque élevé d'inondation à partir de données géographiques et environnementales.

2. Problématique

Comment utiliser les données topographiques, hydrologiques et météorologiques pour identifier les zones à risque d'inondation et produire une carte de vulnérabilité à l'aide de Python ?

3. Objectif général

Développer un modèle numérique sous Python permettant de cartographier automatiquement les zones inondables d'une région donnée.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre les mécanismes des inondations ;
- apprendre les bases des SIG ;
- exploiter des données géographiques ouvertes ;
- traiter des modèles numériques de terrain (MNT) ;
- calculer des indicateurs de risque ;
- produire des cartes de vulnérabilité ;
- développer une application simple de visualisation.

6. Théorie

6.1 Les inondations

Les principales causes sont :

- fortes précipitations ;
- débordement des cours d'eau ;

- urbanisation non planifiée ;
- faible infiltration des sols ;
- obstruction des systèmes de drainage.

6.2 Facteurs influençant les inondations

Les paramètres étudiés seront :

- altitude ;
- pente ;
- distance aux rivières ;
- occupation du sol ;
- intensité des précipitations ;
- type de sol ;
- densité du réseau hydrographique.

6.3 Systèmes d'Information Géographique (SIG)

Les étudiants découvriront :

- les données raster et vecteur ;
- les projections cartographiques ;
- les couches géographiques ;
- les analyses spatiales.

6.4 Modèle Numérique de Terrain (MNT)

Le MNT permet de représenter l'altitude du terrain et de calculer :

- la pente ;
- la direction d'écoulement ;
- les zones de dépression ;
- les bassins versants.

7. Méthodologie

Étape 1 : Recherche bibliographique

Étude des phénomènes d'inondation et des méthodes de cartographie.

Étape 2 : Collecte des données

Les étudiants utiliseront des données ouvertes telles que :

Données topographiques

- Modèle Numérique de Terrain (SRTM ou Copernicus DEM).

Données hydrologiques

- rivières ;
- lacs ;
- bassins versants.

Données météorologiques

- précipitations ;
- température (optionnelle).

Occupation des sols

- zones urbaines ;
- forêts ;
- terres agricoles ;
- plans d'eau.

Étape 3 : Prétraitement

À l'aide de Python et QGIS :

- découpage de la zone d'étude ;

- reprojection des données ;
- nettoyage ;
- harmonisation des résolutions spatiales.

Étape 4 : Calcul des indicateurs

Les étudiants calculeront :

- carte des pentes ;
- carte des altitudes ;
- distance aux cours d'eau ;
- indice topographique d'humidité (Topographic Wetness Index - TWI) ;
- densité hydrographique.

Étape 5 : Modélisation du risque

Deux approches pourront être comparées.

Approche 1 : Analyse multicritère

Chaque facteur recevra un poids selon son importance (méthode AHP, par exemple), puis les couches seront combinées pour produire une carte de risque.

Approche 2 : Machine Learning (optionnelle)

Utilisé des données historiques d'inondation sont disponibles, les étudiants pourront entraîner un modèle (Random Forest, XGBoost) pour prédire les zones à risque.

Étape 6 : Cartographie

Production de cartes thématiques :

- altitude ;
- pente ;
- réseau hydrographique ;
- occupation des sols ;
- risque d'inondation (faible, moyen, élevé).

À la fin du projet, les étudiants fourniront :

- un programme Python ;
- les cartes produites ;
- un rapport scientifique ;
- une présentation PowerPoint.

PROJET 8 : Intelligence artificielle pour prévoir la consommation électrique d'un bâtiment

1. Contexte

La consommation électrique des bâtiments représente une part importante de la demande énergétique mondiale. Une mauvaise anticipation des besoins entraîne :

- une surconsommation ;
- une mauvaise gestion des équipements ;
- des coûts énergétiques élevés ;
- une surcharge des réseaux électriques.

Avec le développement des **compteurs intelligents (Smart Meters)** et de l'**intelligence artificielle (IA)**, il est possible d'analyser les habitudes de consommation et de prévoir la demande électrique future.

Ce projet consiste à développer un modèle d'intelligence artificielle capable de prédire la consommation électrique d'un bâtiment à partir de données historiques et de paramètres environnementaux.

2. Problématique

Comment utiliser l'intelligence artificielle pour prédire la consommation électrique future d'un bâtiment afin d'améliorer sa gestion énergétique ?

3. Objectif général

Développer un modèle prédictif sous Python permettant d'estimer la consommation électrique horaire, journalière ou mensuelle d'un bâtiment.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre les facteurs influençant la consommation électrique ;
- analyser des données énergétiques réelles ;
- appliquer des méthodes de Machine Learning ;
- développer un modèle de prédiction ;
- comparer plusieurs algorithmes ;
- évaluer la précision des prédictions ;
- proposer une stratégie d'optimisation énergétique.

5. Facteurs influençant la consommation électrique

Le modèle pourra utiliser plusieurs variables :

Variables temporelles

- heure de la journée ;
- jour de la semaine ;
- mois ;
- saison ;
- jours fériés.

Variables météorologiques

- température ;
- humidité ;
- vitesse du vent ;
- rayonnement solaire.

Variables liées au bâtiment

- surface ;
- nombre d'occupants ;
- type d'appareils électriques ;
- système de climatisation ;
- éclairage.

6. Connaissances théoriques

6.1 Consommation électrique

La puissance électrique consommée est :

où : P : puissance électrique ; U : tension ; I : courant.

L'énergie consommée pendant une durée :

$$P(t) = U(t) \times I(t)$$

6.2 Intelligence artificielle

Les étudiants étudieront :

- apprentissage supervisé ;
- régression ;

$$E = \int P(t)dt$$

7. Données utilisées

Les étudiants pourront utiliser des bases de données publiques :

1. Données de consommation électrique

Exemples :

- UCI Electricity Load Diagrams Dataset ;
- Smart* Dataset ;
- UK-DALE Dataset

2. Données météorologiques

Sources :

- température ;
- humidité ;
- rayonnement solaire.

8. Méthodologie

Étape 1 : Analyse des données

Utilisation de :

- Pandas ;
- NumPy.

Objectifs :

- comprendre les variations de consommation ;
- détecter les anomalies ;
- visualiser les tendances.

Graphiques :

- consommation journalière ;
- consommation mensuelle ;
- évolution selon la température.

Étape 2 : Prétraitement

Les étudiants effectueront :

- nettoyage des données ;
- suppression des valeurs manquantes ;
- normalisation ;
- création de variables.

Exemple :

À partir de :

Date | Heure | Température | Consommation

Créer :

Jour | Saison | Heure | Consommation future

12. Résultats attendus

Le programme devra permettre :

- de prévoir la consommation horaire ;
- de détecter les périodes de forte demande ;
- d'identifier les équipements énergivores ;
- de proposer des scénarios d'économie.

À la fin du projet, les étudiants fourniront :

- un programme Python ;
- un rapport scientifique ;
- une présentation PowerPoint.

PROJET 9 : Détection précoce des maladies cardiaques

1. Contexte

Les maladies cardiovasculaires constituent l'une des principales causes de décès dans le monde. Une détection précoce permet d'améliorer considérablement les chances de traitement et de réduire les complications.

Grâce aux progrès de l'intelligence artificielle (IA) et de l'apprentissage automatique (Machine Learning), il est désormais possible d'analyser des données médicales afin d'aider les professionnels de santé à identifier les patients présentant un risque élevé de maladie cardiaque. Ce projet consiste à développer un système d'aide à la décision utilisant des algorithmes d'IA pour prédire le risque de maladie cardiaque à partir de données cliniques. Il est important de souligner que ce système **n'établit pas un diagnostic médical**, mais fournit une estimation du risque qui peut soutenir le travail des professionnels de santé.

2. Problématique

Comment développer un modèle d'intelligence artificielle capable d'estimer le risque de maladie cardiaque à partir de données cliniques et biologiques ?

3. Objectif général

Développer un modèle prédictif sous Python permettant d'estimer le risque de maladie cardiaque à partir de données médicales.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre les principaux facteurs de risque cardiovasculaire ;
- explorer une base de données médicale ;
- nettoyer et préparer les données ;
- développer plusieurs modèles de Machine Learning ;
- comparer les performances des modèles ;
- développer une interface simple permettant d'estimer le risque.

5. Facteurs étudiés

Les variables pourront inclure :

Données démographiques

- âge ;
- sexe.

Données cliniques

- pression artérielle ;
- fréquence cardiaque ;
- indice de masse corporelle (IMC) ;
- douleur thoracique (catégories).

Données biologiques

- cholestérol ;
- glycémie.

Examens complémentaires (selon le jeu de données)

- résultats ECG ;
- angine d'effort ;
- pente du segment ST.

Variable cible

- présence ou absence d'une maladie cardiaque (classification binaire).

6. Connaissances théoriques

Les étudiants étudieront :

Les maladies cardiovasculaires

- maladie coronarienne ;
- infarctus du myocarde ;
- insuffisance cardiaque ;
- facteurs de risque.

Intelligence artificielle

- apprentissage supervisé ;
- classification ;
- validation croisée ;
- métriques de performance.

Prétraitement des données

- valeurs manquantes ;
- normalisation ;
- encodage des variables catégorielles ;
- équilibrage des classes (optionnel).

8. Méthodologie

Étape 1 : Recherche bibliographique

Étude :

- des maladies cardiaques ;
- des facteurs de risque ;
- des méthodes d'intelligence artificielle.

Étape 3 : Prétraitement

- suppression des doublons ;
- traitement des valeurs manquantes ;
- normalisation ;
- séparation des données en ensembles d'entraînement et de test.

Étape 4 : Développement des modèles

Plusieurs modèles seront comparés.

Modèle 1

Régression logistique

Modèle 2

Arbre de décision

Modèle 3

Random Forest

Résultats attendus

Le système devra :

- estimer le risque de maladie cardiaque à partir des données saisies ;
- comparer les prédictions de plusieurs modèles ;
- afficher un score de risque accompagné d'un niveau de confiance ;
- Un rapport synthétique destiné à un usage pédagogique.

À la fin du projet, les étudiants fourniront :

- un programme Python ;
- un rapport scientifique ;
- une présentation PowerPoint.

PROJET 10 : Simulation de la production de biogaz à partir des déchets agricoles

1. Contexte

La gestion des déchets agricoles constitue un défi majeur dans de nombreux pays, notamment en Afrique. Les résidus de culture (maïs, riz, coton), les déjections animales et les déchets agroalimentaires sont souvent sous-exploités, alors qu'ils représentent une ressource importante pour la production d'énergie renouvelable.

La méthanisation est un procédé biologique qui transforme la matière organique en biogaz, principalement composé de méthane (CH_4) et de dioxyde de carbone (CO_2), grâce à l'action de micro-organismes en absence d'oxygène.

Ce projet propose de développer un simulateur sous Python permettant de prédire la production de biogaz à partir de différents types de déchets agricoles et d'étudier l'influence des paramètres de fonctionnement d'un digesteur.

2. Problématique

Comment modéliser et simuler la production de biogaz à partir de déchets agricoles afin d'optimiser les conditions de méthanisation ?

3. Objectif général

Développer un modèle numérique sous Python permettant de simuler la production de biogaz et de méthane à partir de différents déchets agricoles.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre le principe de la méthanisation ;
- étudier les facteurs influençant la production de biogaz ;
- développer un modèle mathématique de production ;
- simuler différents scénarios de fonctionnement ;
- analyser les performances du digesteur ;
- proposer des conditions optimales de production.

5. Compétences développées

Les étudiants apprendront à :

- programmer en Python ;
- résoudre des équations différentielles simples ;
- analyser des données expérimentales ;
- visualiser des résultats scientifiques ;
- réaliser une étude de sensibilité des paramètres.

6. Connaissances théoriques

6.1 La méthanisation

La méthanisation est un processus biologique comprenant quatre étapes principales :

- hydrolyse ;
- acidogenèse ;
- acétogenèse ;
- méthanogenèse.

Le produit final est un mélange gazeux appelé biogaz.

Composition typique :

- méthane (CH_4) : 50 à 70 % ;
- dioxyde de carbone (CO_2) : 30 à 50 % ;
- traces de H_2S , NH_3 et vapeur d'eau.

6.2 Déchets agricoles utilisables

Les étudiants pourront comparer plusieurs substrats :

- fumier bovin ;
- fumier de volaille ;
- lisier de porc ;
- résidus de maïs ;
- paille de riz ;
- bagasse de canne à sucre ;
- coques de cacao ;
- déchets de manioc.

6.3 Facteurs influençant la production

Le modèle prendra en compte :

- température du digesteur ;
- pH ;
- temps de séjour hydraulique (HRT) ;
- concentration en matière organique ;
- rapport carbone/azote (C/N) ;
- humidité.

7. Modélisation mathématique

Une approche simplifiée consiste à utiliser une cinétique du premier ordre :

$$\frac{dS}{dt} = -kS$$

où : S : concentration du substrat ; k : constante de dégradation.

La production cumulée de biogaz peut être estimée par :

$$B(t) = B_{\max} (1 - e^{-kt})$$

avec : B(t) : biogaz produit à l'instant t ; B_{max} : production maximale théorique.

Une extension possible est le **modèle de Gompertz modifié**, largement utilisé pour décrire la production cumulative de biogaz.

8. Méthodologie

Étape 1 : Recherche bibliographique

Étude :

- de la digestion anaérobie ;
- des déchets agricoles ;
- des modèles cinétiques de production.

Étape 2 : Collecte des données

Les étudiants pourront utiliser :

- des données expérimentales publiées ;
- des jeux de données ouverts ;
- des valeurs issues de la littérature scientifique.

Les données incluront :

- type de déchet ;
- masse introduite ;
- température ;

- pH ;
- temps de digestion ;
- volume de biogaz produit.

Étape 3 : Développement du modèle

Le programme devra :

- calculer la dégradation du substrat ;
- estimer la production journalière de biogaz ;
- calculer la quantité de méthane produite ;
- représenter les courbes de production.

Étape 4 : Simulations

Les étudiants feront varier :

- la température (25–55 °C) ;
- le temps de digestion ;
- la quantité de substrat ;
- le rapport C/N ;
- le type de déchet.

Étape 5 : Analyse de sensibilité

Évaluer l'effet de chaque paramètre sur :

- la production totale de biogaz ;
- le rendement en méthane ;
- le temps nécessaire pour atteindre la production maximale.

Résultats attendus

Le programme devra produire :

- l'évolution de la concentration du substrat ;
- la production journalière de biogaz ;
- la production cumulée de biogaz ;
- le volume de méthane produit ;
- l'influence de la température et du temps de digestion.

À la fin du projet, les étudiants fourniront :

- un programme Python ;
- un rapport scientifique ;
- une présentation PowerPoint.

PROJET 11 : Simulation du trafic routier par automates cellulaires

1. Contexte

La croissance rapide des villes entraîne une augmentation du trafic routier, des embouteillages, de la consommation de carburant et des émissions de gaz à effet de serre. Une meilleure compréhension de la dynamique du trafic est essentielle pour améliorer la mobilité urbaine et concevoir des systèmes de transport plus efficaces.

Les **automates cellulaires** sont une méthode simple et puissante pour modéliser des systèmes complexes où des comportements globaux émergent de règles locales. Ils sont largement utilisés pour simuler le trafic routier, notamment avec le modèle de **Nagel-Schreckenberg**, qui décrit le déplacement des véhicules sur une route discrétisée.

Dans ce projet, les étudiants développeront un simulateur de trafic routier sous Python afin d'étudier la formation des embouteillages et l'impact de différents paramètres sur la fluidité de la circulation.

2. Problématique

Comment modéliser la circulation des véhicules à l'aide d'automates cellulaires afin d'étudier les embouteillages et d'améliorer la fluidité du trafic ?

3. Objectif général

Développer un simulateur numérique sous Python permettant de reproduire la circulation routière et d'analyser l'influence de différents paramètres sur les performances du trafic.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre les principes des automates cellulaires ;
- modéliser une route sous forme de grille discrète ;
- simuler le déplacement des véhicules ;
- analyser les phénomènes d'embouteillage ;
- étudier l'influence de la densité du trafic et de la vitesse maximale ;
- comparer différents scénarios de circulation.

6. Connaissances théoriques

6.1 Les automates cellulaires

Un automate cellulaire est constitué de :

- une grille de cellules ;
- des états (vide ou occupé par un véhicule) ;
- des règles de transition ;
- une évolution discrète dans le temps.

6.2 Modèle de Nagel-Schreckenberg

Chaque véhicule est caractérisé par :

- sa position ;
- sa vitesse.

À chaque pas de temps, les règles suivantes sont appliquées :

1. **Accélération** : si la vitesse est inférieure à la vitesse maximale, elle augmente de 1.
2. **Freinage** : si un véhicule est trop proche du suivant, sa vitesse est réduite pour éviter une collision.
3. **Ralentissement aléatoire** : avec une probabilité ppp, le conducteur ralentit d'une unité de vitesse.
4. **Déplacement** : le véhicule avance selon sa vitesse finale.

Ce modèle permet de reproduire des phénomènes observés dans le trafic réel, comme les embouteillages spontanés.

6.3 Paramètres étudiés

Les simulations feront varier :

- longueur de la route ;
- nombre de véhicules ;
- vitesse maximale ;
- probabilité de ralentissement ;
- durée de la simulation.

7. Méthodologie

Étape 1 : Recherche bibliographique

Étude :

- des modèles de trafic routier ;
- des automates cellulaires ;
- du modèle de Nagel-Schreckenberg.

Étape 2 : Développement du modèle

Les étudiants représenteront une route sous forme d'un tableau :

- une case vide ;
- une case occupée par un véhicule.

Chaque véhicule sera caractérisé par :

- sa position ;
- sa vitesse.

Étape 3 : Développement de l'algorithme

Le programme appliquera les règles du modèle à chaque pas de temps :

- accélération ;
- freinage ;
- ralentissement aléatoire ;
- déplacement.

Étape 4 : Simulations

Les étudiants compareront plusieurs scénarios :

Cas 1

Trafic faible.

Cas 2

Trafic moyen.

Cas 3

Trafic dense.

Cas 4

Influence de la vitesse maximale.

Cas 5

Influence d'un ralentissement aléatoire.

Étape 5 : Analyse

Les étudiants calculeront :

- débit de circulation (véhicules/minute) ;
- vitesse moyenne ;
- longueur moyenne des embouteillages ;
- temps moyen de parcours ;
- densité du trafic.

Résultats attendus

Le simulateur devra permettre :

- d'observer l'évolution du trafic ;
- de visualiser les embouteillages ;
- de mesurer les performances du réseau routier ;
- de comparer plusieurs scénarios.

Les résultats seront présentés sous forme de graphiques :

- vitesse moyenne en fonction de la densité ;
- débit de circulation ;
- évolution du nombre de véhicules ;

- temps de parcours.

À la fin du projet, les étudiants fourniront :

- un programme Python ;
- un rapport scientifique ;
- une présentation PowerPoint.

PROJET 12 : Détection automatique des maladies des plantes par vision artificielle

1. Contexte

L'agriculture joue un rôle essentiel dans l'économie de nombreux pays, notamment en Afrique. Cependant, les maladies des plantes entraînent chaque année des pertes importantes de rendement et de qualité des cultures. Une détection tardive peut favoriser la propagation des maladies et augmenter les coûts liés aux traitements.

Les progrès de la **vision artificielle** et de l'**intelligence artificielle (IA)** permettent aujourd'hui de détecter automatiquement certaines maladies à partir de photographies des feuilles, des fruits ou des tiges. Ces outils peuvent aider les agriculteurs à intervenir plus rapidement et à améliorer la gestion des cultures.

Dans ce projet, les étudiants développeront une application Python capable de reconnaître automatiquement certaines maladies végétales à partir d'images. Ce système est destiné à fournir une **aide au diagnostic**, sans remplacer l'expertise d'un agronome ou d'un phytopathologiste.

2. Problématique

Comment utiliser la vision artificielle et l'intelligence artificielle pour identifier automatiquement des maladies des plantes à partir d'images numériques ?

3. Objectif général

Développer un système intelligent sous Python capable de détecter et de classifier des maladies des plantes à partir d'images.

4. Objectifs spécifiques

Les étudiants devront :

- comprendre les principales maladies des plantes ;
- constituer ou utiliser une base de données d'images ;
- prétraiter les images ;
- développer un modèle de classification ;
- comparer plusieurs algorithmes d'intelligence artificielle ;
- évaluer les performances du système ;
- développer une interface utilisateur simple.

5. Cultures étudiées

Le projet pourra porter sur une ou plusieurs cultures, par exemple :

- maïs ;
- tomate ;
- manioc ;
- riz ;
- cacao ;
- café ;
- bananier ;
- pomme de terre.

6. Maladies étudiées

Selon la culture et le jeu de données choisi, les étudiants pourront détecter :

Tomate

- mildiou ;
- alternariose ;
- taches bactériennes ;
- septoriose.

Maïs

- rouille commune ;
- cercosporiose ;
- brûlure des feuilles.

Manioc

- mosaïque du manioc ;
- striure brune.

Cacao

- pourriture brune des cabosses ;
- maladie du balai de sorcière (selon les données disponibles).

Étape 4 : Développement des modèles

Approche 1 : Machine Learning

Extraction de caractéristiques :

- histogrammes de couleur ;
- texture ;
- contours ;
- HOG (Histogram of Oriented Gradients).

Puis classification avec :

- SVM ;
- Random Forest ;
- KNN.

Approche 2 : Deep Learning

Les étudiants entraîneront un CNN ou utiliseront le **transfert d'apprentissage** avec un modèle pré-entraîné (par exemple MobileNet ou ResNet).

Résultats attendus

Le système devra être capable de :

- reconnaître une plante saine ou malade ;
- identifier la maladie parmi les classes disponibles ;
- afficher la probabilité de chaque classe ;
- un rapport simple pour l'utilisateur.

À la fin du projet, les étudiants fourniront :

- un programme Python ;
- un rapport scientifique ;
- une présentation PowerPoint.